BIG DATA SECURITY AND ANALYSIS USING BLOCKCHAIN AND AI IN IOT

Prakash Tekchandani
PhD Research Scholar
Roll No: 2020801004

Indranil Pradhan
M.Tech CSIS
Roll No: 2019202008

Date of Submission: 16-Nov-2020

Abstract

Internet of thing (IoT) and mobile devices generates massive amount of Big data through sensors. This data is used to build intelligent applications through machine learning. To build these applications data is collected from devices into data centers for training machine learning models. Usually the training of models is performed on central server, this approach requires the transfer of data from devices to central server. This centralized training approach is not efficient because users are much less likely to share data to centralized data centers due to privacy issues and bandwidth limitations. We proposed novel secured federated learning approach and blockchain to securely train the model locally on devices and store the model and its parameters in blockchain for traceability and immutability.

Keywords: Federated Learning, IoT, Blockchain, Privacy and Data Security.

Section 1: Introduction

The rapidly increasing volume and complexity of Big Data [1] has led to the significant growth of data. With the fast evolution of technology, enormous amount of data is produced from IoT devices, healthcare, manufacturing, banking. In Big Data, we extract knowledge and value from various forms of data. This field encompasses statistics, data analysis and other advanced methods used to understand and analyze actual processes using data. Organizations are embracing the power of big data, using powerful analytics for driving strategic business decisions, identify opportunities, and speed up performance. At the same time, big data security concerns rise with the vast increase in data usage and utilization. Finally, the big data adoption comes down to one question for many enterprises: How can we use big data's potential while effectively reducing security risks?

Big data is primarily dependent on infrastructure, cloud and edge computing [2],[3] which impose data security risks. Untrusted applications, which lacks security measure defined by standard authority and data governance, can easily impose risks into enterprise. Not only this, increase usage of mobile and IoT devices to work from anywhere at any time utilizing the cloud network adds to the threat of big data security. Additionally, depth of the user's access to sensitive data is another concern in terms of security. The top preference for any enterprise must include defining the

secured privilege access for users. Well defined security protocols and measures must be defined to avoid potential misuse of data keeping in mind that some users have access to highly sensitive data in some business processes. This is commonly referred to as the "principle of least privilege". As the big data keeps expanding, the risk of data sharing and data source authentication is another point of discussion. Parties involving data sharing do not completely trust on each other during the data transaction. For example, government has a fear of violation in national security and is more cautious about data sharing as well as individual face privacy concern about their personal information. Furthermore, multi-stakeholder involvement has resulted in poor control over data transfer which creates the hinderance in development of the big data industry.

Big Data collected from IOT devices and other application is stored in data centers for training machine learning models to build intelligent applications. Centralized training is not efficient because users are much less likely to share data to centralized data centers due to privacy issues and bandwidth limitations.

We proposed secured federated learning and blockchain to train the model, data privacy and storing the model and its parameters in blockchain. In federated learning model training, IOT devices learn collectively from a shared model. The server trains the shared model with proxy data and send model to IOT devices, devices improves the model through local or federated data training. The updated trained model is summarized before sending back to server. Now the improved model updates are ready for uploads to server.

To make federated learning secured we proposed a novel approach to authenticate device and server before any uploads starts. After authentication between device and server the model is send to the server, the server aggregates the model to obtain an improved combined model. The exchange of models between device and server is continued for several iterations until superior model is obtained. In federated learning the model is passed to training data, instead of training data to model. By this the device has full control on data and achieves data privacy. The improved model and is parameters are stored in blockchain for traceability and immutability.

Section 2: Literature Review

Uchi *et. al.* [4] proposed Big Data security through blockchain access control. In this work, they focused on Big data security and privacy issues in cloud computing environment. They proposed Blockchain Role-Based Access Control to address security in big data. In their study they

modeled blockchain using Hyperledger composer. In Role-Based Access Control, access is granted on a role-by- role basis. They created smart contracts to access, grant and verify users based on the roles assigned.

LIYe *et. al.* [5] proposed safe circulation of big data resources through blockchain. In their proposed work there are three entities as data producers, users and blockchain network. Smart contract keeps track of large amount of data generated by data producers and store the data in blockchain. Smart contract is also used by users to obtain authorized data from blockchain.

Chirstan Espostio *et. al.* [6], proposed blockchain technology for healthcare data in cloud. New block is created as soon as the patient data is generated. This block is distributed among all peers patient network. After the consensus and proof of work the block is inserted in blockchain. This enables global view of patient history. This view is immutable and traceable, and cryptographically linked for security. If the consensus among peers is not reached, then the block is declared orphan.

Ascribe [7], this project deals with the intellectual property data. They are implementing blockchain to provide ownership layer so that the original owner of the art is compensated.

Chang et al. [8] proposed an incentive based Blockchain for edge computing. They used Stackelberg equilibrium to find optimal incentive to miners to participate in process of blockchain and obtain computational resources from Edge service provider. Jangirala et al., 2019 [9] proposed lightweight blockchain enabled communication and computation protocol (LBRAPS) for authentication. This protocol is RFID based and used for manufacturing supply chain in 5G mobile edge computing environment. LBRAPS is XOR, one way hash and bitwise rotation based protocol. It included authentication and key agreement. This protocol is efficient in computation and supports security features which are efficient against active attacks. Yang et al. (2020) [10] proposed distributed blockchain approach for privacy of network topology for Mobile edge computing (MEC) in 5G. They implemented mutual networking trust, heterogeneous MEC and collaborative routing through Blockchain. There scheme improved the efficiency of MEC in 5G.

Lu et al. [11] proposed privacy during data sharing through Federated Learning and Blockchain. They mainly focused on minimizing the consensus work in blockchain through federated learning. The data model is shared instead of actual data to achieve privacy of data. Chai et al. [12] proposed hierarchical blockchain enabled federated learning for Internet of Vehicles (IoV). They also used game theory for distributed knowledge sharing. They aggregate the knowledge in

hierarchal layers and perform correlation. Their approach improved the security of knowledge sharing in IoV.

Section 3: Proposed Scheme

3.1 Motivation

Data is everywhere and sensor equipped IOT devices are major source of data. Mobiles devices connected with internet generates data on every click and intelligent applications are required for enhanced user experience.

First motivation for the proposed approach is to make user data available for learning without breaching the data privacy.

Secondly, to create customized model for user device, to achieve this model should be trained on local data of user device.

3.2 Proposed Approach:

We used secured federated learning and blockchain to train the model, data privacy and storing the model and its parameters in blockchain. In federated learning model training, IOT devices learn collectively from a shared model. The server trains the shared model with proxy data and send model to IOT devices, devices improves the model through local or federated data training. The updated trained model is at device end and sent to server. Now the server aggregates the received model with the model present in server and test on test data set. If the accuracy on test dataset reaches the threshold value set by server, then the aggregated model is set to be kept in blockchain.

To make federated learning secured we proposed a novel approach to authenticate device and server before any uploads starts. After authentication between device and server the model is send to the server, the server aggregates the model to obtain an improved combined model. The exchange of models between device and server is continued for several iterations until superior model is obtained. In federated learning the model is passed to training data, instead of training data to model. By this the device has full control on data and achieves data privacy. The improved model and is parameters are stored in blockchain for traceability and immutability.

Below are the steps followed in our proposed approach and pictorially described in figure 1:

1. Model with random weights or model trained with very small dataset, is distributed to all the devices i.e. from central server to A,B,C as shown in figure 1.
2. Each device (A, B, C) has its own dataset, and the model received from central server. Each device now train model locally on respective end. Authentication is established between participant and server for secured federated learning.
3. During the training the model on the individual participant dataset, the model weights are updated and the updated model weights is then sent to the central server.
4. The central servers collect the model from each device and aggregates the weights.

   $$Aggregated\ Model\ Parameters = \frac{Central\ Server\ Model\ Parameters + Parameters\ from\ Devices}{2}$$

   Step 1,2,3,4 is repeated until the model is trained.
5. After the model is trained, transaction is requested to add model in blockchain. The block containing transaction is created and send to each participant in the network. Participants validates the transactions through proof of work and block is added to the existing blockchain and transaction is complete.

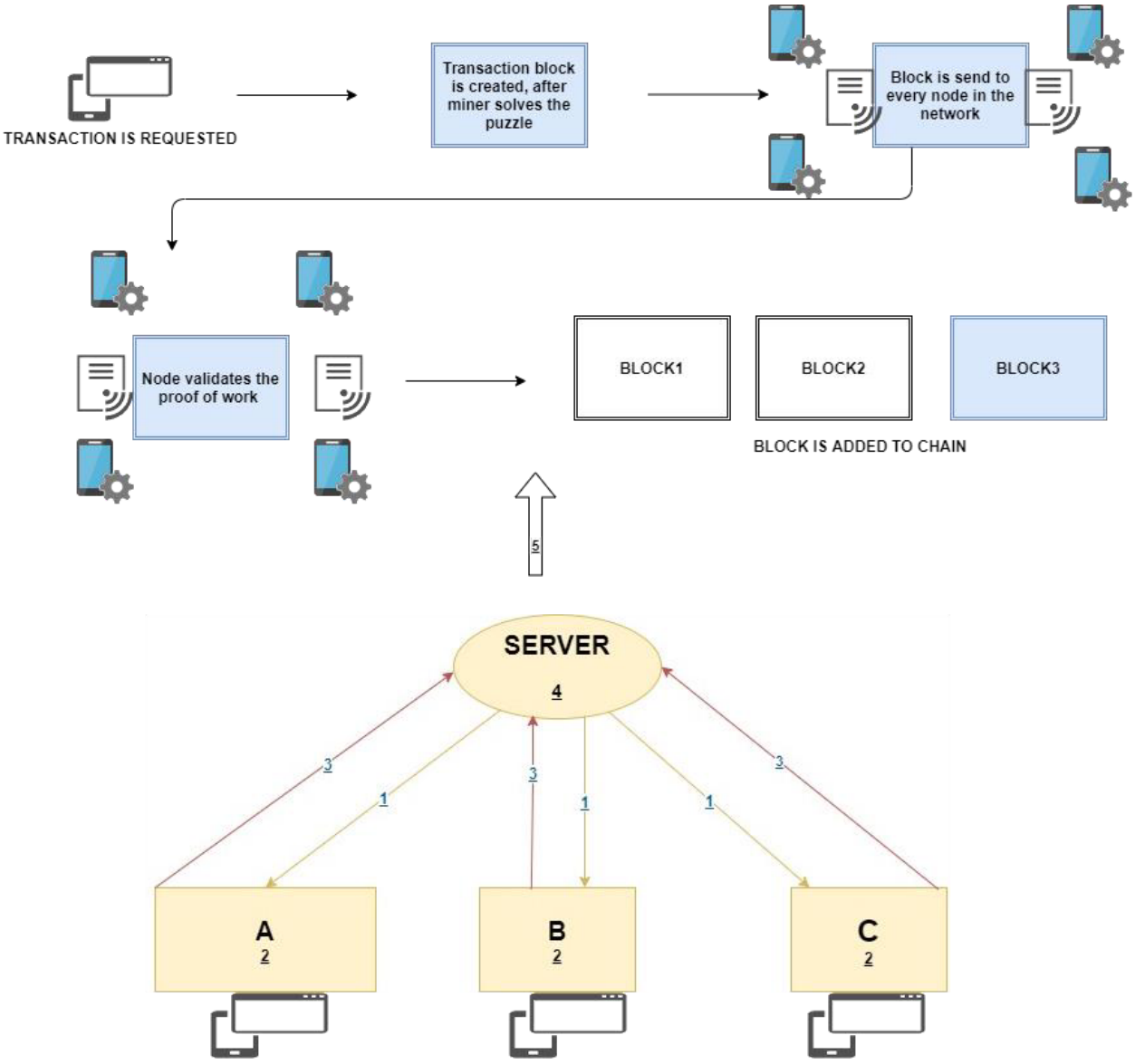


Figure 1: Proposed Scheme

3.3 Mutual Authentication & Key Establishment

Whenever device has data to train and is ready with its model it connects to server for aggregation of model. While connecting to server it needs to authenticate itself to server by

solving the challenge from the server. And after solving the challenge the session key gets established between the server and for that client.
This helps from attack, confidentiality, data integrity from the adversary.

Notations:

Symbol	**Description**
ID_C	Identity of Device
T_S	Timestamp at Server
T_C	Timestamp at Device
R_C , R_C' , R_N , R_N'	Nonce
H_{func}	Hash Function
M_1 , M_2 , M_3	Message
LR(X,Y)	Left rotate X by hamming distance Y
RR(X,Y)	Right rotate X by hamming distance Y
H_{val} , H_{val}'	Hashed value
X XOR Y	X Exclusive Or Y
HAM(X,Y)	Hamming distance between X and Y
X ||Y	X concatenates Y
SK	Session Key

3.3.1 Steps to Mutual Authentication and Key Establishment

1. The device and server agree upon the same public cryptographic hash function, symmetric encryption method. The server has stored the identity of all the devices along with the hash value of the identity in a dictionary format where hash value of identity is the key and identity of the device is the value.
2. When device has data to train and its model is ready, the device tries to communicate to the server by sending the hash of the ID_C to the server through the public channel.

$$H_{ID} = H_{func}(ID_C)$$

3. The server receives the H_{ID} from the device. Server checks if the H_{ID} is present in its database or not. If present, server fetches the corresponding ID_C of the H_{ID}. If not, present server discards the connection.
4. To ensure that the H_{ID} has been sent by the device, the server challenges device. As a challenge the server generates R_N, current timestamp T_s and perform the following calculations

$$M_1 = LR((R_N \text{ XOR } ID_C \text{ XOR } T_S), HAM(ID_C, T_S))$$

And to ensure the integrity of message server calculates the Hash in the following manner

$$Hash_1 = Hash_{func}(M_1 \,||ID_C\, ||R_N\, ||T_S)$$

And server sends $\{M_1, Hash_1, T_S\}$ message to the device.

5. Device receives the message $\{M_1, Hash_1, T_S\}$ from the server. Device first calculates the current timestamp T_{cur} and checks

$$T_{cur} - T_S \leq \Delta T$$

If above condition is true, then device accepts the message otherwise discards the message.

6. After checking the timestamp, device calculates the challenge given from the server using the following calculation

$$R_N' = RR(M_1, HAM(ID_C, T_S)) \text{ XOR } ID_C \text{ XOR } T_S$$

After getting the RN' the device calculates the Hash value.

$$Hash_1' = Hash_{func}(M_1 \,||ID_C\, ||R_N'\, ||T_S)$$

After calculating $Hash_1'$, the device compares this with received $Hash_1$ from the server.
If $Hash_1'$ == $Hash_1$ the device is sure that it has solved the challenge the successfully. Hence,

$$R_N = R_N'$$

Else, i.e. - $Hash_1'$!= $Hash_1$ the device disconnects the connection.

7. Before key establishment, it verifies the server and let the server know that it has solved the challenge. To accomplish this, the device generates a new nonce, R_C and take the current timestamps T_C and does the following calculation-

$$M_2 = LR((R_N \text{ XOR } ID_C \text{ XOR } T_C), HAM(ID_C, T_C))$$

$$M_3 = LR((R_N \text{ XOR } ID_C \text{ XOR } R_C), HAM(R_N, ID_C))$$

And to maintain the integrity of message device also calculates the Hash value in the following manner-

$$Hash_2 = Hash_{func}(M_2 \| M_3 \| R_N \| T_C \| R_C)$$

And device send the message $\{M_2, M_3, Hash_2, T_C\}$ to the server.

8. The server receives the message from the device. At first, server calculates the current timestamp T_{cur} and checks-

$$T_{cur} - T_C \leq \Delta T$$

If the above condition satisfies then server accepts the message.
Otherwise, server discards the message.

9. Then server calculates R_N' from the message received from the device using the following calculation-

$$R_N' = RR(M_2, HAM(ID_C, T_C)) \text{ XOR } ID_C \text{ XOR } T_C$$

After getting R_N' it checks with previous R_N.
If $R_N == R_N'$ server get assured that device has successfully solved the challenge.
Else i.e. - $R_N != R_N'$ server disconnects the connection.

10. After verifying the nonce, server calculates the nonce generated by device using the following calculation –

$$R_C' = RR(M_3, HAM(R_N, ID_C)) \text{ XOR } R_N \text{ XOR } ID_C$$

Then calculates the Hash value in the following manner –

$$Hash_2' = Hash_{func}(M_2 \| M_3 \| R_N \| T_C \| R_C')$$

After calculating $Hash_2'$, the server checks this with received $Hash_2$ from the device.
If $Hash_2'$ == $Hash_2$ the server be sure that it has successfully derived the nonce from the device. So,

$$R_C = R_C'$$

Else, i.e. - $Hash_2'$! = $Hash_2$ the server disconnects the connection.

11. To let device know that server has successfully solved the challenge random nonce R_C , server calculates $Hash_3$ and current time stamp T_S as-

$$Hash_3 = Hash_{func}(R_C \| R_N)$$

And server sends {$Hash_3$, T_S} to device.

12. The device receives the message from the server. At first, device calculates the current timestamp T_{cur} and checks-

$$T_{cur} - T_s \leq \Delta T$$

If the above condition satisfies then server accepts the message.
Otherwise, server discards the message.

13. Then device calculates the $Hash_3'$ as-

$$Hash_3' = Hash_{func}(R_C \| R_N)$$

And checks if $Hash_3'$ == $Hash_3$, If successful then go to step 13 else disconnect the connections.

14. After completing above steps, the device and server establishes the session key between

them in the following manner-

Server -

$$SK = H_{func}(ID_C \ || R_C)$$

Device -

$$SK = H_{func}(ID_C \ || R_C)$$

In future all the communication between server and device using this session key through symmetric encryption method.

3.3.2 Flow Diagram of Mutual Authentication and Key Establishment

Device **Server**

Calculate
$H_{ID} = H_{func}(ID_C)$

Send H_{ID}

Server verifies H_{ID} and fetches corresponding ID_C.
Server calculates –

$M_1 = LR((R_N \text{ XOR } ID_C \text{ XOR } T_S), HAM(ID_C, T_S))$

$Hash_1 = Hash_{func}(M_1 \ || ID_C \ || R_N \ || T_S)$

Send $\{M_1, Hash_1, T_S\}$

Device Checks

$T_{cur} - T_S \leq \Delta T$ If success,

Device calculates –

$R_N' = RR(M_1, HAM(ID_C, T_S)) \text{ XOR } ID_C \text{ XOR } T_S$

$Hash_1' = Hash_{func}(M_1 \ || ID_C \ || R_N' \ || T_S)$

If $Hash_1' == Hash_1$ then $R_N = R_N'$

Then device generates R_C and T_C and calculates-

$M_2 = LR((R_N \text{ XOR } ID_C \text{ XOR } T_C), HAM(ID_C, T_C))$

$M_3 = LR((R_N \text{ XOR } ID_C \text{ XOR } R_C), HAM(R_N, ID_C))$

$Hash_2 = Hash_{func}(M_2 \ || M_3 \ || R_N \ || T_C \ || R_C)$

Send $\{M_2, M_3, Hash_2, T_C\}$

Server checks-

$T_{cur} - T_C \leq \Delta T$

Server calculates-

$R_N' = RR(M_2, HAM(ID_C, T_C))$ XOR ID_C XOR T_C

And checks $R_N == R_N'$ and if success,

Server Calculates-

$R_C' = RR(M_3, HAM(R_N, ID_C))$ XOR R_N XOR ID_C

$Hash_2' = Hash_{func}(M_2 \| M_3 \| R_N \| T_C \| R_C')$

Checks $Hash_2' == Hash_2$ and if success,

$R_C = R_C'$

Server Calculates $Hash_3 = Hash_{func}(R_N \| R_C)$

Send $\{Hash_3, T_S\}$

Device Checks

$T_{cur} - T_S \leq \Delta T$ and If success,

Device calculates $Hash_3' = Hash_{func}(R_N \| R_C)$

And checks if $Hash_3' == Hash_3$ and If Successful,

Device Calculates Session key

$SK = H_{func}(ID_C \| R_C)$

Server Calculates Session key-

$SK = H_{func}(ID_C \| R_C)$

Section 4: Analysis of Our proposed Scheme

4.1 Security Analysis

1. **Confidentiality**

During the message exchange between devices and server the messages $M_1 = \{R_N, ID_C, T_S\}$, $M2 = \{R_N, ID_C, T_C\}$ and $M3 = \{R_N, ID_C, R_C\}$ messages are formed with random nonce R_N , R_C . It is not possible for the adversary to guess the random nonce. The confidentiality during transmission of messages is intact. The confidentiality is preserved.

2. **Device Impersonation Attack**

Let one device using its identity tries to read another device's message. But it is not possible as for each different device the generated random nonce is different. For each device, generated M_1, M_2, M_3 is different because of R_N, R_C. So, the device cannot read other's message.

3. **Forward secrecy**

Let one device leaves the connection and get captured by adversary after that. The adversary cannot communicate to the server through session key because in every session there is newly generated session key and used random nonces i.e. R_N , R_C are newly generated in each session.

4. **Data Poisoning**

Let one device wants to distort the trained model with the bad datasets. But when the device sends its own model for aggregation at server, the server explicitly checks the model performance using its own test data set. And if the model performs better than the threshold value then only model gets aggregated.

5. **Mutual Authentication and Session key**

At first, the devices send their identity hashed by one way hash function H_{func}. If server doesn't find any entry for that particular hash value in its database, then it disconnects the connection. Later to confirm that the hashed identity has been received to server from

authentic device, the server generates challenge for the device using device's original identity ID_C and random nonce R_N and send the user message M1 = {R_N, ID_C, T_S} by using XOR and rotation using hamming distance and provide the message integrity. Server also send the hash value of {M_1, ID_C, R_N, T_S} to the device. If the device is authentic and it solves the challenge using this $R_N' = RR(M_1, HAM(ID_C, T_S)) \text{ XOR } ID_C \text{ XOR } T_S$. After that to let server know that device has solved the challenge successfully it also sends message M_2 = {R_N, ID_C, T_C } and device also wants to verify if it receives the message from the actual server and not from any adversary. So, device generate one random nounce R_C and message M_3 = {R_N, ID_C, R_C} to server. And if server is authentic and it solves the challenge using this $R_C' = RR(M_3, HAM(R_N,ID_C)) \text{ XOR } R_N \text{ XOR } ID_C$ and also checks if device has successfully solved its challenge using $R_N' = RR(M_2, HAM(ID_C, T_C)) \text{ XOR } ID_C \text{ XOR } T_C$ and after this server let client about that it has solved the challenge given by device successfully by sending $Hash_3$ = {$R_N || R_C$} and device also checks the $Hash_3$ after this both calculates the session key = $H_{func}(ID_C || R_C)$, this is how mutual authentication is completed.

6. **Data Integrity during authentication-**

During the transmission of communications every message has been transferred while keeping the integrity of message intact by calculating hash of the messages in the form of $Hash_1$, $Hash_2$, $Hash_3$.

7. **Anonymity and Untraceable** –

The hash function H_{func} is one way and it is not reversible. This is computationally infeasible for the adversary to guess the ID_C from the hash value. The ID_C is XORed within the message and after that rotation is applied based on random time stamps and random nonces. It impossible to get the ID_C from the messages and get the session key. And new time stamps and random nonces in each session makes user anonymous and untraceable for the adversary.

8. **Ephemeral Secret Leakage Attack**

Suppose the secrets like R_N, R_C is leaked to the adversary. Then when adversary tries to

solve the challenge from the server it cannot solve the challenge as it doesn't know about the identity of device ID_C which is securely stored at device and in the database of server.

9. **Replay Attack**

As all the communication is going on public channel, let adversary read the messages M_1, M_2, M_3 and does the replay attack by replaying these messages multiple times to the server. But it does not get successful as the random nonce and time stamp is different every time.

10. **Man in the middle Attack-**

Let there is adversary in between server and device. And adversary receives the conversation between the device and server. While the adversary tries to modify the messages M_1, M_2 or M_3, it is not successful because adversary do not know about the random nonce R_N, R_C or the original identity of the device ID_C . This prevents man-in-the-middle attack.

4.2 Storage Analysis

1. In our approach the device does not send the data to the server to store it in blockchain. Instead server sends only the model which gets stored in the blockchain. This reduces the overhead of storing huge dataset for training of AI/ML model.
2. For every end device the server stores the identities of the device along with the hash values of the identities of the devices which is necessary to authenticate the devices. If there are 10million devices, then it would need 560MB of data storage to store the devices identities with its hashed value which incredibly low.
3. In every session the devices and server stores only the session key. After establishing the session key all the random nonce and time stamp values are deleted from both the device and server both.

4.3 Communication Analysis

1. All the communications have been done over the public channel.

2. Every time, during authentication and key establishment phase to maintain the message integrity, the hash values i.e. $Hash_1$, $Hash_2$, $Hash_3$ have been shared. If it is found that message has been modified after comparing with hash value, it immediately disconnects the connection.
   After establishing session-key the communication has been over the encryption and decryption of the messages between the server and device.
3. At first the hashed identity of device takes 64 bytes. The size of M_2, M_3, M_4 messages is 3 bytes. The size of $Hash_1$, $Hash_2$ and $Hash_3$ is each of size 64 bytes. The session key established between device and server is of 16 bytes size.
4. The model sent from the device to the server for aggregation is of size 56 bytes.

4.4 Computation Analysis-

1. During the mutual authentication and key establishment phase the used methods are XOR of two strings, calculating hash of string and rotation of string is fast with less computation and complexity.
2. During model training the Genetic Algorithm Neural Networks has been used as a classifier. Currently the classifier is used as a binary classifier. It also can be used as multiclass classifier.
3. The communication between server and device happens using symmetric key encryption and decryption which takes very low time.
4. The time taken to establish the session key between server and device 0.03978 seconds.
5. To train model at device end took around 73 seconds on average for our dataset.
6. There 3 left rotations, 3 right rotation, 5 times hash function involved in during the establishment of session key.

Section 5: Comparative Study

Our proposed scheme has been compared with "*Blockchain and Federated Learning for Privacy-preserved Data Sharing in Industrial IoT*" by Yunlong Lu et al.[14] and "*Blockchain-based Node-aware Dynamic Weighting Methods for Improving Federated Learning Performance*" by

You Jun Kim et al.[13].

5.1 Security Analysis

Mutual authentication between end device, server and secure model transfer between them is not performed by Yunlong Lu et al and You Jun Kim et al. Table1 lists the security comparisons between Yunlong Lu et al, You Jun Kim et al. and our proposed scheme.

	Yunlong Lu et al. Scheme	You Jun Kim et al. scheme	Our scheme
Confidentiality	**NO**	**NO**	**YES**
Anonymity and Untraceability	**NO**	**NO**	**YES**
Mutual Authentication and session key	**NO**	**NO**	**YES**
Forward Secrecy	**NO**	**NO**	**YES**
Device Impersonation Attack	**NO**	**NO**	**YES**
Replay Attack	**NO**	**NO**	**YES**
Man in the middle Attack	**NO**	**NO**	**YES**
Ephemeral Secret Leakage Attack	**NO**	**NO**	**YES**
Data Integrity during Authentication	**NO**	**NO**	**YES**
Data Poisoning	**YES**	**YES**	**YES**
Blockchain	**YES**	**YES**	**YES**
Encrypted Model Store in Blockchain	**YES**	**NO**	**YES**

Table 1: Comparison Study

5.2 AI/ML models

	Yunlong Lu et al. Scheme	You Jun Kim et al. scheme	Our Scheme
Model	**Regression Model**	**Multilayer Perceptron**	**Binary classifier using Genetic Algorithm Neural Network**

Section 6: Blockchain Implementation

6.1 Block Structure

header	
block_version	Unique Id of the block
previous_hash	Hash of the previous Block
timestamp	The time when block is created
block_owner	The owner of the block. For our case the owner is the hashed identity of the device whose model last successfully reaches the accuracy threshold value on the test data of the server.
proof	Proof of work by miner
merkle_tree	The root hash of the merkle tree.
payload	
current_hash	The hash of the entire current block
sign	The signature on the block i.e. the hash of hashed identity of the device and model parameter such as $Sign = Hash_{func}(Hash_{ID} \| model_parameter)$
transaction	The encrypted aggregated model

6.2 Steps in Blockchain Implementation

1. When an aggregated model achieves accuracy threshold value set by the server, the aggregated model is set for mining.
2. The model which will be stored in the blockchain is encrypted by session key of the server and the device.
3. To maintain the model integrity, the hash of the hashed identity of device and model parameter is kept as the signature.
4. Miners compete with each other to solve the cryptographic challenge as part of proof of work.
5. As soon as the miner solve the challenge, the block is created. In our proposed scheme the challenge is to obtain an integer for which the first four leading zeroes are obtained by

hash of the difference of the square of previous block proof and the square of current block proof.

6. The block is distributed to participating nodes in blockchain to get added.
7. Participating nodes verify the proof of work. If it is a valid proof of work, then the block is valid otherwise the block is declared orphan.
8. After the validation of proof of work, the signature gets verified with hashed identity of the device and model parameter to verify that model is not tampered in between the processes.
9. If signature is verified, the new block is added to blockchain.

Section 7: Future research Directions:

Currently our proposed is using the binary classifier in federated learning. In future work we will enhance the proposed scheme to use multiclass classifier in federated learning.

REFERENECES

[1] Liang F, Yu W, An D, Yang Q, Fu X, Zhao W. A survey on big data market: Pricing, trading and protection. IEEE Access. 2018 Feb 16;6:15132-54.
[2] Shi W, Cao J, Zhang Q, Li Y, Xu L. Edge computing: Vision and challenges. IEEE internet of things journal. 2016 Jun 9;3(5):637-46.
[3] Yu W, Liang F, He X, Hatcher WG, Lu C, Lin J, Yang X. A survey on the edge computing for the Internet of Things. IEEE access. 2017 Nov 29;6:6900-19.
[4] Uchibeke UU, Schneider KA, Kassani SH, Deters R. Blockchain access control Ecosystem for Big Data security.
[5] Yue L, Junqin H, Shengzhi Q, Ruijin W. Big data model of security sharing based on blockchain. In2017 3rd International Conference on Big Data Computing and Communications (BIGCOM) 2017 Aug 10 (pp. 117-121). IEEE.
[6] Esposito C, De Santis A, Tortora G, Chang H, Choo KK. Blockchain: A panacea for healthcare cloud-based data security and privacy?. IEEE Cloud Computing. 2018 Mar 28;5(1):31-7.
[7] McConaghy T, Holtzman D. Towards an ownership layer for the internet. ascribe GmbH. 2015 Jun 24.
[8] Chang, Zheng & Guo, Wenlong & Guo, Xijuan & Zhou, Zhenyu. (2020). Incentive Mechanism for Edge-Computing-Based Blockchain. IEEE Transactions on Industrial Informatics. PP. 1-1. 10.1109/TII.2020.2973248.
[9] Jangirala, Srinivas & Das, Ashok Kumar & Vasilakos, Athanasios. (2019). Designing Secure Lightweight Blockchain-Enabled RFID-Based Authentication Protocol for Supply Chains in 5G Mobile Edge Computing Environment. IEEE Transactions on Industrial Informatics. 10.1109/TII.2019.2942389.
[10] Yang, Hui & Liang, Yongshen & Yuan, Jiaqi & Yao, Qiuyan & Yu, Ao & Zhang, Jie. (2020). Distributed Blockchain-Based Trusted Multidomain Collaboration for Mobile Edge Computing in 5G and Beyond. IEEE Transactions on Industrial Informatics. PP. 1-1. 10.1109/TII.2020.2964563.
[11] Lu, Yunlong & Huang, Xiaohong & Dai, Yueyue & Maharjan, Sabita & Zhang, Yan. (2019). Blockchain and Federated Learning for Privacy-Preserved Data Sharing in Industrial IoT. IEEE Transactions on Industrial Informatics. PP. 1-1. 10.1109/TII.2019.2942190.
[12] Chai, Haoye & Leng, Supeng & Chen, Yijin & Zhang, Ke. (2020). A Hierarchical Blockchain-Enabled Federated Learning Algorithm for Knowledge Sharing in Internet of Vehicles. IEEE Transactions on Intelligent Transportation Systems. PP. 1-12. 10.1109/TITS.2020.3002712.
[13] Y. J. Kim and C. S. Hong, "Blockchain-based Node-aware Dynamic Weighting Methods for Improving Federated Learning Performance," 2019 20th Asia-Pacific Network Operations and Management Symposium (APNOMS), Matsue, Japan, 2019, pp. 1-4, doi: 10.23919/APNOMS.2019.8893114.
[14] Y. Lu, X. Huang, Y. Dai, S. Maharjan and Y. Zhang, "Blockchain and Federated Learning for Privacy-Preserved Data Sharing in Industrial IoT," in IEEE Transactions on Industrial Informatics, vol. 16, no. 6, pp. 4177-4186, June 2020, doi: 10.1109/TII.2019.2942190.